



Alma Mater Studiorum-Università di Bologna

Scuola di Ingegneria

Fondamenti di Informatica T2 **Esercitazione: Battaglia Navale**

Corso di Laurea in Ingegneria Informatica

Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

La Battaglia Navale

- La *battaglia navale* si gioca su una griglia NxN (solitamente N=10)
 - righe e colonne sono numerate da 1 (a volte dall'alto, a volte dal basso)
 - spesso per le colonne si usano lettere A-J
 - ATTENZIONE all'ordine di righe e colonne: nella prima forma, ogni cella è espressa da R1C2, R4C5, etc. (quindi, prima la riga); nella seconda, le stesse celle risultano «B1», «E4», etc. (quindi, prima la colonna)
- Ogni giocatore dispone sulla griglia le proprie *navi*, di vario tipo
 - ogni tipo di nave ha la sua dimensione

PORTAEREI										
INCROCIATORI										
TORPEDINIERE										
SOMMERGIBILI										

	A	B	C	D	E	F	G	H	I	J
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										

La Battaglia Navale

- **Scopo del gioco** è colpire le navi avversarie, sparando un colpo per volta in una data *posizione*
 - se in quella posizione non c'è nulla, l'avversario risponde «ACQUA»
 - se invece c'è una nave, risponde «COLPITO» e la nave si considera danneggiata
 - se il colpo danneggia l'ultima parte integra della nave, la risposta è «AFFONDATA»
- Di norma, il gioco è fra *due giocatori*
 - ognuno ha quindi il suo schema di navi e cerca di indovinare quello avversario
- Nel nostro caso, invece, sarà un **solitario**
 - solo il computer avrà uno schema
 - il giocatore cercherà di vincere nel minor numero possibile di tentativi

	A	B	C	D	E	F	G	H	I	J
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										

Battaglia Navale: vincoli

- *Vincolo di non sovrapposizione*
 - due navi non possono sovrapporsi
- *Vincolo di non adiacenza*
 - due navi non possono essere adiacenti
 - NB: non tutti lo adottano!
 - senza, il gioco è più difficile per il giocatore e più semplice da implementare
→ 😊 *noi non lo adotteremo* 😊
- *Vincolo di non debordare*
 - (ovvio) nessuna nave deve debordare dal campo di gioco

	A	B	C	D	E	F	G	H	I	J
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										



Analisi del problema: entità

- Cerchiamo di identificare le principali *entità* in gioco
 - si parla di navi di un certo «tipo» → occorre modellare il *tipo di nave*
 - si parla di «navi» → occorre modellare la nozione di *nave*
 - si parla di «posizioni» → occorre modellare la nozione di *posizione*
 - si parla di «tabella» → occorre modellare la nozione di *campo di gioco*
- Aspetti da chiarire (il diavolo è nei dettagli...😊)
 - quando parliamo di «nave», cosa intendiamo? Non il tipo (c'è già!), ma *una nave specifica, collocata nel campo di gioco in una specifica posizione e con uno specifico orientamento (orizzontale o verticale)*
 - emerge un ulteriore concetto: l'*orientamento*

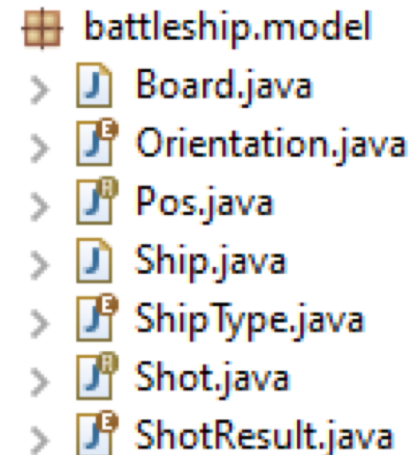
Analisi del problema: entità

- E quando spariamo?

- si dice che si «spara un colpo» verso una certa *posizione*
→ occorre modellare il concetto di *colpo (sparo)*
- l'avversario «risponde» in tre possibili modi («acqua», «colpito», «affondato») → occorre modellare la nozione di *risultato del colpo*

- Riassumendo, abbiamo *sette entità*:

- tipo di nave → *ShipType*
- nave → *Ship*
- posizione → *Pos*
- orientamento → *Orientation*
- campo di gioco → *Board*
- colpo → *Shot*
- risultato del colpo → *ShotResult*



Analisi del problema: vincoli

- I vincoli determinano il grosso dell'algoritmica:
 - *Vincolo di non debordare*: una nave non può debordare dal campo
 - *Vincolo di non sovrapposizione*: due navi non possono sovrapporsi
- Come esprimerli?
 - in linea di principio, *tramite vincoli matematici* sulle posizioni delle navi e sul loro orientamento
 - MA, soprattutto per la non sovrapposizione, ciò potrebbe diventare *molto complicato e tricky...*
 - Qualche idea alternativa?

	A	B	C	D	E	F	G	H	I	J
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										

Analisi del problema: algoritmica

- Vincolo di non debordare**

- siano (r,c) la posizione della prima cella della nave e len la sua lunghezza
 → l'ultima posizione è $(r+len-1,c)$ se la nave è verticale,
 oppure $(r, c+len-1)$ se è orizzontale
- vincolo: $r \geq 0, c \geq 0$ e inoltre, a seconda dell'orientamento:
 - se verticale → $r+len-1 \leq size, c \leq size$
 - se orizzontale → $r \leq size, c + len-1 \leq size$

- Vincolo di non sovrapposizione**

- Caso A: due navi orizzontali (o verticali)
 → l'ultima cella della prima nave non deve sovrapporsi alla prima della seconda nave
- Caso B: una nave orizzontale e una verticale
 → nessuna cella della prima nave deve sovrapporsi a nessuna cella della seconda nave

	A	B	C	D	E	F	G	H	I	J
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										

Analisi del problema: algoritmica

- Esprimere matematicamente il vincolo di non sovrapposizione porta, purtroppo, a dover *distinguere molti casi e sotto-casi*:
 - Premessa: siano $(r1, c1)$ la posizione della prima cella della nave, $len1$ la sua lunghezza e analogamente $(r2, c2)$ e $len2$ per la seconda nave
 - Caso A: se la nave#1 *precede* la nave#2, **per non sovrapporsi** la cella finale di nave#1 deve *precedere strettamente* la prima cella di nave#2
 - se verticale $\rightarrow r1 + len1 - 1 < r2, c1 = c2$
 - se orizzontale $\rightarrow r1 = r2, c1 + len1 - 1 < c2$
 - Caso A': se la nave#1 *segue* la nave #2, le coordinate delle due navi si scambiano:
 - se verticale $\rightarrow r2 + len2 - 1 < r1, c1 = c2$
 - se orizzontale $\rightarrow r1 = r2, c2 + len2 - 1 < c1$
 - Caso B: una nave orizzontale e una verticale (...segue...)

	A	B	C	D	E	F	G	H	I	J
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										

Analisi del problema: algoritmica

- Esprimere matematicamente il vincolo di non sovrapposizione porta, purtroppo, a dover *distinguere molti casi e sotto-casi*:
 - Premessa: siano $(r1, c1)$ la posizione della prima cella della nave, $len1$ la sua lunghezza e analogamente $(r2, c2)$ e $len2$ per la seconda nave
 - Caso B: una nave orizzontale e una verticale: per non sovrapporsi
 - nave#1 orizzontale, nave #2 verticale
 - $r1 < r2 \ || \ r1 > r2 + len2 - 1 \ ||$
 $(r2 \leq r1 \leq r2 + len2 + 1 \ \&\&$
 $(c2 < c1 \ || \ c2 > c1 + len1 - 1))$
 - nave#2 orizzontale, nave #1 verticale
 - coordinate scambiate
 - $r2 < r1 \ || \ r2 > r1 + len1 - 1 \ ||$
 $(r1 \leq r2 \leq r1 + len1 + 1 \ \&\&$
 $(c1 < c2 \ || \ c1 > c2 + len2 - 1))$

	A	B	C	D	E	F	G	H	I	J
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										



Analisi del problema: codifica

- Tradurre queste specifiche in codice *così come sono* porterebbe sicuramente a **codice estremamente contorto**, pieno di if /switch, con molti/troppi casi e sotto-casi → **illeggibile e immanutibile**
- Anche predisponendo opportune librerie di ausilio per
 - incapsulare il calcolo dell'ultima cella di una nave
 - catturare tutte le disuguaglianze fra posizioni di riga/colonna in opportuni metodi di utilità dal nome «parlante»

il risultato sarebbe comunque molto complicato e sgradevole

- **estrema dipendenza da micro-dettagli = approccio error-prone**
- basta un $<$ al posto di un $\leq$, un -1 mancante, etc. e nulla funziona più
→ debug allucinante, tempi di sviluppo incerti



Analisi del problema: codifica

```
public boolean overlaps(Pos pos) {
    return switch (this.getOrientation()) {
        case HORIZONTAL -> this.getPos().isRowEqualTo(pos) &&
            pos.isColumnGreaterThanOrEqualTo(this.getPos()) && pos.isColumnLessThanOrEqualTo(this.getLastPos());

        case VERTICAL -> this.getPos().isColumnEqualTo(pos) &&
            pos.isRowGreaterThanOrEqualTo(this.getPos()) && pos.isRowLessThanOrEqualTo(this.getLastPos());
    };
}
```

```
public boolean overlaps(Ship other) {
    if (this.isOrientationEqualTo(other)) {
        return switch (this.getOrientation()) {
            case HORIZONTAL -> this.getPos().isRowEqualTo(other.getPos())
                &&
                (this.getPos().isColumnLessThanOrEqualTo(other.getPos()) ? this.getLastPos().isColumnGreaterThanOrEqualTo(other.getPos())
                    : other.getLastPos().isColumnGreaterThanOrEqualTo(this.getPos()));

            case VERTICAL -> this.getPos().isColumnEqualTo(other.getPos())
                &&
                (this.getPos().isRowLessThanOrEqualTo(other.getPos()) ? this.getLastPos().isRowGreaterThanOrEqualTo(other.getPos())
                    : other.getLastPos().isRowGreaterThanOrEqualTo(this.getPos()) );
        };
    }
    else {
        return switch (this.getOrientation()) {
            case HORIZONTAL -> (other.getPos().isColumnGreaterThanOrEqualTo(this.getPos()) && other.getPos().isColumnLessThanOrEqualTo(this.getLastPos()))
                &&
                (this.getPos().isRowGreaterThanOrEqualTo(other.getPos()) && this.getPos().isRowLessThanOrEqualTo(other.getLastPos()));

            case VERTICAL -> (this.getPos().isColumnGreaterThanOrEqualTo(other.getPos()) && this.getPos().isColumnLessThanOrEqualTo(other.getLastPos()))
                &&
                (other.getPos().isRowGreaterThanOrEqualTo(this.getPos()) && other.getPos().isRowLessThanOrEqualTo(this.getLastPos()));
        };
    }
}
```

Possibile, ma... *un incubo!*



Analisi del problema: revisione

- «Forse» esprimere quei vincoli in termini di disuguaglianze fra insiemi di righe e/o colonne *non è una buona idea* 😊
 - qualche alternativa?
- **Regola aurea: quando il codice si fa troppo contorto è segno che occorre *elevare il livello di astrazione***
 - servono nuovi concetti
 - *e/o nuovi metodi*
 - *e/o nuove entità (classi, etc.)*
- Alcune possibili idee
 - arricchire il concetto di posizione: non solo (riga,colonna), ma un'entità capace di *capire se sia esterna o interna* a un dato intervallo
 - rivedere la rappresentazione della nave: non più solo (riga,colonna) di inizio e fine, ma un *insieme di posizioni occupate*

Analisi del problema: revisione

- Esprimere una nave come *insieme di posizioni occupate* cambia lo scenario
 - **Vincolo di non sovrapposizione** = assenza di posizioni comuni fra quelle di due navi, *indipendentemente da come esse siano disposte* 😊

```
public boolean overlaps(Pos pos) {  
    return Pos.contains(this.getPositions(), pos);  
}  
  
public boolean overlaps(Ship other) {  
    return Pos.findCommons(this.getPositions(), other.getPositions()).length>0;  
}
```

- **Vincolo di non debordare**: le posizioni non eccedono il campo

```
// 1) controllo range posizione iniziale  
if (ship.getPos().exceeds(size, "Posizione fuori range")) return false;  
  
// 2) controllo range posizione finale  
if (ship.getLastPos().exceeds(size, "Lunghezza nave eccessiva per " + ship.toString() + ": pos
```



Progetto: chi fa cosa

- La posizione
 - incapsula le *coordinate* (riga, colonna)
 - è in grado di *calcolare la «prossima» posizione* a distanza k celle
 - sa dire se *eccede* un campo di gioco di una certa dimensione (size)
 - fornisce *operatori di confronto* ($<$, $\leq$, $>$, $\geq$, $=$) per riga e per colonna
 - incapsula la libreria statica per array di posizioni:
 - *findCommons* per trovare le posizioni comuni a due collezioni di posizioni
 - *contains* per sapere se una collezione di posizioni contiene una certa posizione data
- La nave
 - è caratterizzata da posizione, tipo (lunghezza), integrità, orientamento, nome
 - sa dire la *collezione (array) di posizioni che occupa*
 - sa dire *se si sovrappone* a una posizione o a un'altra nave
 - può *decrementare la propria integrità* (se colpita), rispondendo «colpita» o «affondata» come necessario

Progetto: chi fa cosa

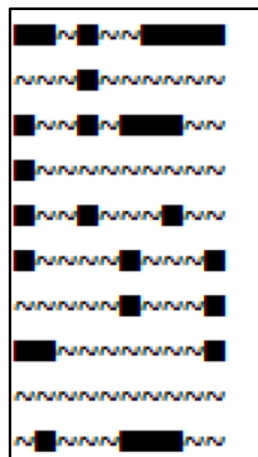
- Il campo di gioco
 - viene costruito di una certa dimensione e riempito con un insieme di navi
→ durante la fase di riempimento, si controlla che ogni nave rispetti i vincoli
 - registra i colpi sparati e, se qualche nave viene colpita, ne riduce l'integrità
 - può mostrare lo stato attuale (dei colpi sparati) in forma pseudo-grafica
 - sa dire in ogni momento *quante navi* sono ancora operative (non affondate)
 - può rivelare la lista completa delle navi (anche in forma pseudo-grafica)
- Lo sparo
 - esprime semplicemente la coppia (posizione colpo, risultato del colpo)
- Il main gestisce la logica di gioco
 - costruisce il campo di gioco e lo popola di navi
 - iterativamente, chiede all'utente che mossa vuole fare e, in risposta, mostra lo stato del campo di gioco che ne risulta
 - Il gioco termina quando tutte le navi sono scoperte (o quando l'utente scrive END)



- ■ ■

L'esempio di riferimento

- Campo di gioco configurato con queste 12 navi
 - 2 portaerei (lunghezza 4)
 - 4 incrociatori (lunghezza 3)
 - 4 cacciatorpedinieri (lunghezza 2)
 - 3 sommergibili (lunghezza 1)



	A	B	C	D	E	F	G	H	I	J
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										

E ora.. al lavoro!

- Progettare, riflettere, codificare!

- tipo di nave → *ShipType*
- nave → *Ship*
- posizione → *Pos*
- orientamento → *Orientation*
- campo di gioco → *Board*
- colpo → *Shot*
- risultato del colpo → *ShotResult*

